

INDEX

Experiment No	Experiment Name	Page No
1	Write a program to draw some points	
2	Write a program to draw line using slope intercept method.	
3	Write a program to draw a line using Bresenham algorithm.	
4	Write a program to show translation transformation.	
5	Write a program to show rotation transformation.	
6	Write a program to draw a circle using Bresenham algorithm.	
7	Write a program to show scaling transformation.	
8	Write a program to show mouse operation.	
9	Write a program to show arm motion.	
10	Write a program to draw ellipse using Bresenham algorithm.	
11	Write a program to draw a polygon.	
12	Write a program to show the line triangle transformation.	

Signature

1. Write a program to draw some points

```
#include<windows.h>

#include<GL/glut.h>

void myInit(void)
{
    glClearColor(1.0, 1.0, 1.0, 0.0); //background color is white
    glColor3f(0.0f, 0.0f, 0.0f); //drawing color is black
    //initialize view (simple orthographic projection)
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(-1.0,1.0,-1.0, 1.0, -1.0, 1.0);
    //set the viewing coordinates
    gluOrtho2D(0.0, 640.0, 0.0, 480.0);
}

void myDisplay(void)
{
    glClear(GL_COLOR_BUFFER_BIT); //clear the screen
    glPointSize(20.0);
    glBegin(GL_POINTS);
    {
        glVertex2i(20,10);
        glVertex2i(50,10);
    }
}
```

```

    glVertex2i(20,80);
    glVertex2i(50,80);
}

glEnd();

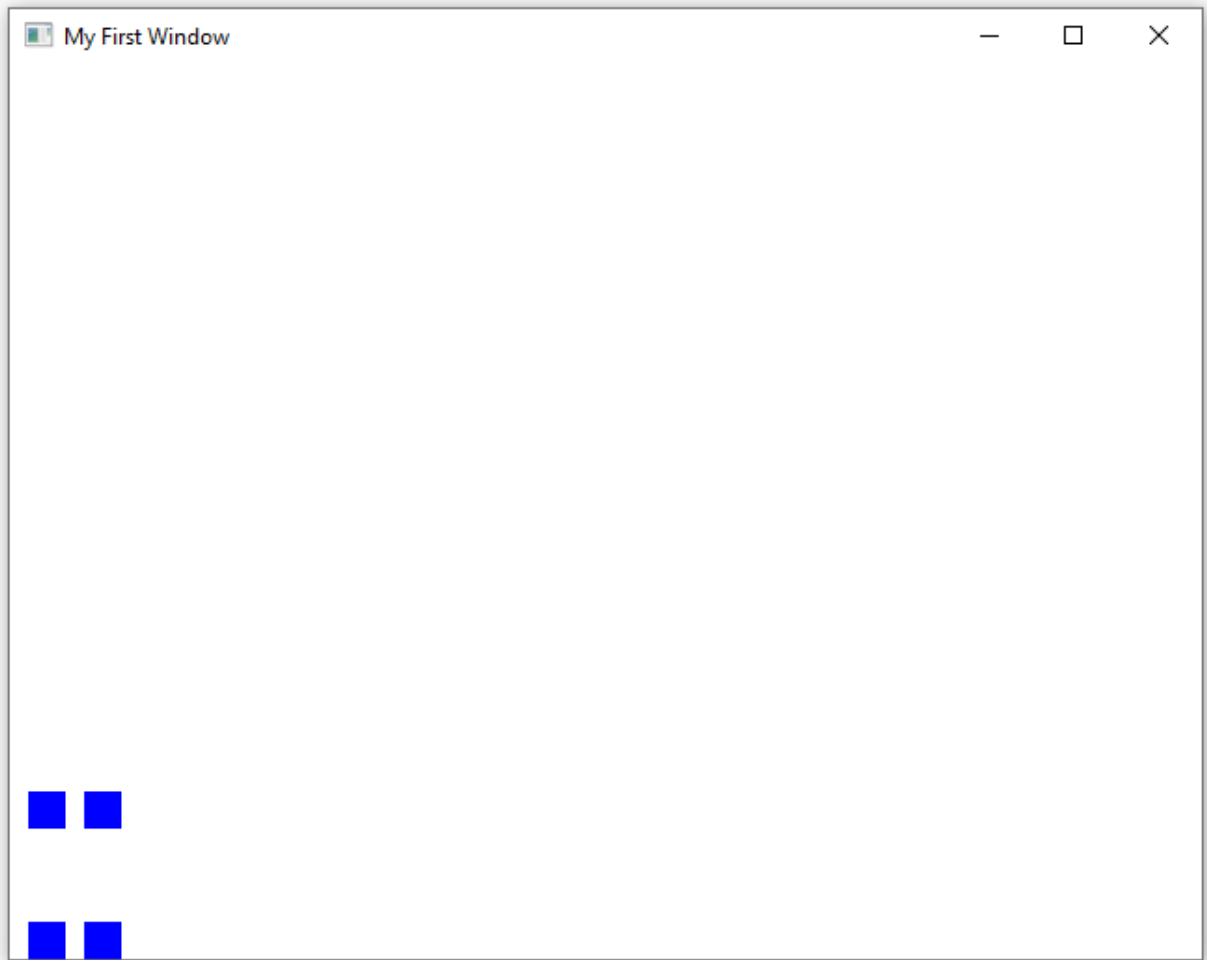
glFlush(); //send all output to display
}

int main(int argc, char** argv)
{
    glutInit(&argc, argv); //initialize the toolkit
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB); //set the display mode
    glutInitWindowSize(640,480); //set window size
    //set window position on screen
    glutInitWindowPosition(100,150);
    //open the screen window and set the name
    glutCreateWindow("My First Window");
    //register the callback functions
    glutDisplayFunc(myDisplay);
    //glutMouseFunc(myMouse);
    myInit();
    glutMainLoop(); //go into a perpetual loop

    return 1;

}

```



2. Write a program to draw line using slope intercept method.

```
#include<windows.h>

#include<math.h>

#include<GL/gl.h>
#include<GL/glu.h>
#include<GL/glut.h>

void myInit(void)
{
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glColor3f(0.0f, 0.0f, 255.0f);
    glPointSize(4.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(-1.0, 1.0, -1.0, 1.0, -1.0, 1.0);
    gluOrtho2D(0.0, 640.0, 0.0, 480.0);
}

void myDisplay(void)
{
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(2.0);
    GLint x,y, x1=50, x2=100, y1=100, y2=200;
```

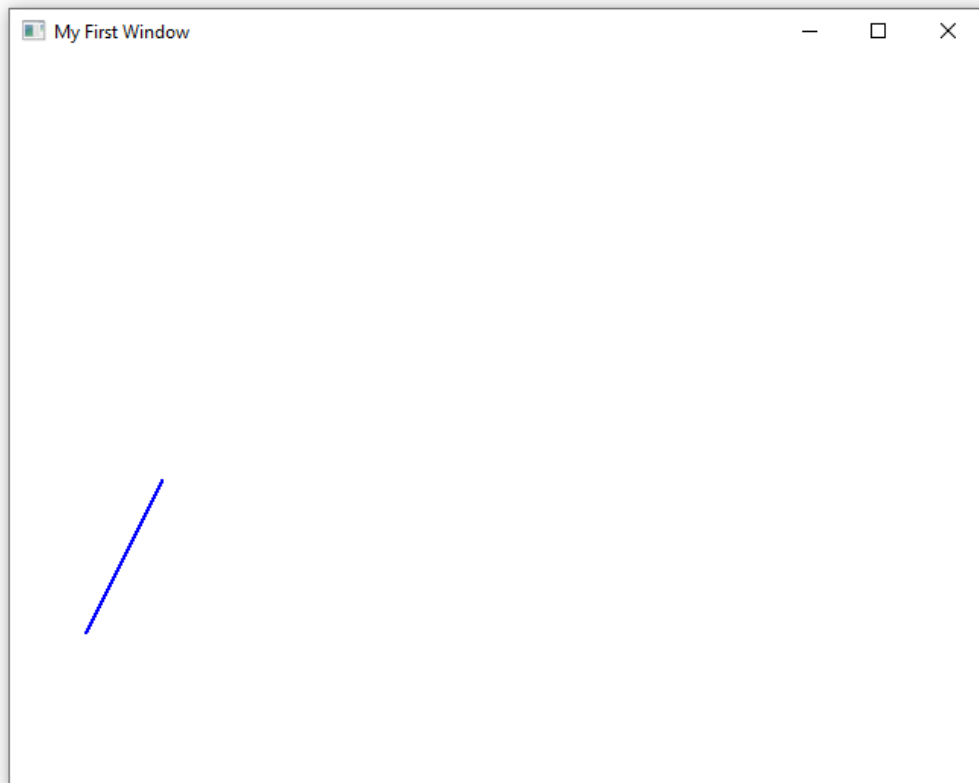
```
GLdouble xf, m, b;  
x=x1; y=y1;  
m=(y2-y1)/(x2-x1);  
b=y1-m*x1;  
glBegin(GL_POINTS);  
{  
    glVertex2i(x,y);  
}  
glEnd();
```

```
while(y<y2)  
{  
    y++;  
    xf=(y-b)/m;  
    x=floor(xf+0.5);  
    glBegin(GL_POINTS);  
    {  
        glVertex2i(x,y);  
    }  
    glEnd();  
}
```

```
glFlush();  
}
```

```
int main(int argc, char** argv)
```

```
{  
    glutInit(&argc, argv);  
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);  
    glutInitWindowSize(640, 480);  
  
    glutInitWindowPosition(100,150);  
    glutCreateWindow("My First Window");  
    glutDisplayFunc(myDisplay);  
  
    myInit();  
    glutMainLoop();  
  
    return 1;  
}
```



3. Write a program to draw a line using Bresenham algorithm.

```
#include<windows.h>

#include<GL/glut.h>

void myInit(void)
{
    glClearColor(1.0, 1.0, 1.0, 0.0); //background color is white
    glColor3f(255.0f, 0.0f, 0.0f); //drawing color is black
    glPointSize(4.0); //a dot is 2 by 2 pixels
    //glLineWidth(4.0); //a line is 4 times thicker

    //initialize view (simple orthographic projection)
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(-1.0,1.0,-1.0, 1.0, -1.0, 1.0);
    //set the viewing coordinates
    gluOrtho2D(0.0, 640.0, 0.0, 480.0);
}

void myDisplay(void)
{
    glClear(GL_COLOR_BUFFER_BIT); //clear the screen
    // glRecti(10,10,150,60); //draw a rectangle with background color
```

```
//Line drawing algorithm  
glPointSize(2.0);  
  
GLint x,y, x1=50, x2=100, y1=100, y2=200;  
GLint dx, dy, inc1, inc2, d;  
  
x = x1;  
y = y1;  
  
dx = x2-x1;  
dy = y2-y1;  
  
inc1 = 2 * dy;  
inc2 = 2 * (dy-dx);  
  
d = inc1 - dx;  
  
while(x <= x2)  
{  
  
    glBegin(GL_POINTS);  
    {  
        glVertex2i(x,y);  
    }  
  
    glEnd();
```

```

    x++;

    if(d < 0)
    d = d + inc1;

    else {
        d = d + inc2;
        y++;
    }

}

glFlush(); //send all output to display
}

int main(int argc, char** argv)
{
    glutInit(&argc, argv); //initialize the toolkit
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB); //set the display mode
    glutInitWindowSize(640,480); //set window size
    //set window position on screen
    glutInitWindowPosition(100,150);
    //open the screen window and set the name

```

```
glutCreateWindow("My First Window");

glutDisplayFunc(myDisplay);

myInit();
glutMainLoop(); //go into a perpetual loop

//fprintf(stderr, "Hello!\n");
return 1;

}
```



4. Write a program to show translation transformation.

```
#include<windows.h>
#include <GL/glut.h>
#include <stdlib.h>
void init(void)
{
    glClearColor (0.0, 0.0, 0.0, 0.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(-50.0, 50.0, -50.0, 50.0, -1.0, 1.0);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}
void draw_triangle(void)
{
    glBegin (GL_LINE_LOOP);
    glVertex2f(0.0, 25.0);
    glVertex2f(25.0, -25.0);
    glVertex2f(-25.0, -25.0);
    glEnd();
}
void display(void)
{
    glClear (GL_COLOR_BUFFER_BIT);
    glColor3f (1.0, 1.0, 1.0);
```

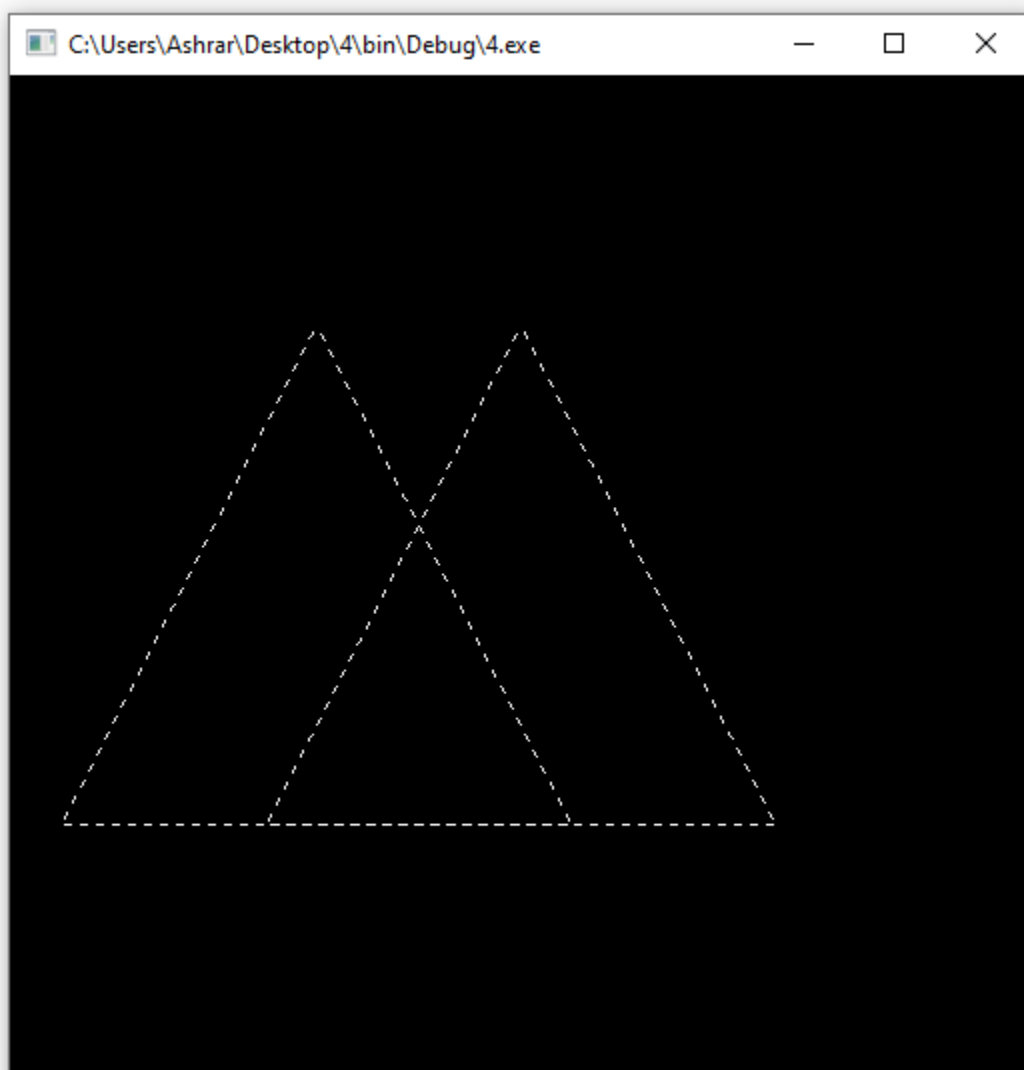
```

glLoadIdentity ();
glColor3f (1.0, 1.0, 1.0);
draw_triangle ();

glEnable (GL_LINE_STIPPLE);
glLineStipple (1, 0xF0F0);
glLoadIdentity ();
glTranslatef (-20.0, 0.0, 0.0);
draw_triangle ();
glFlush ();
}

int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize (500, 500);
    glutInitWindowPosition (100, 100);
    glutCreateWindow (argv[0]);
    init ();
    glutDisplayFunc(display);
    //glutReshapeFunc(reshape);
    // glutKeyboardFunc (keyboard);
    glutMainLoop();
    return 0;
}

```



5. Write a program to show rotation transformation.

```
#include<windows.h>
#include <GL/glut.h>
#include <stdlib.h>

void init(void)
{
    glClearColor (0.0, 0.0, 0.0, 0.0);
    //glShadeModel (GL_FLAT);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    glOrtho(-50.0, 50.0, -50.0, 50.0, -1.0, 1.0);
    glMatrixMode(GL_MODELVIEW);
    glLoadIdentity();
}

void draw_triangle(void)
{
    glBegin (GL_LINE_LOOP);
    glVertex2f(0.0, 25.0);
    glVertex2f(25.0, -25.0);
    glVertex2f(-25.0, -25.0);
    glEnd();
}

void display(void)
```



```

{
    glClear (GL_COLOR_BUFFER_BIT);
    glColor3f (1.0, 1.0, 1.0);

    glLoadIdentity ();
    glColor3f (1.0, 1.0, 1.0);
    draw_triangle ();

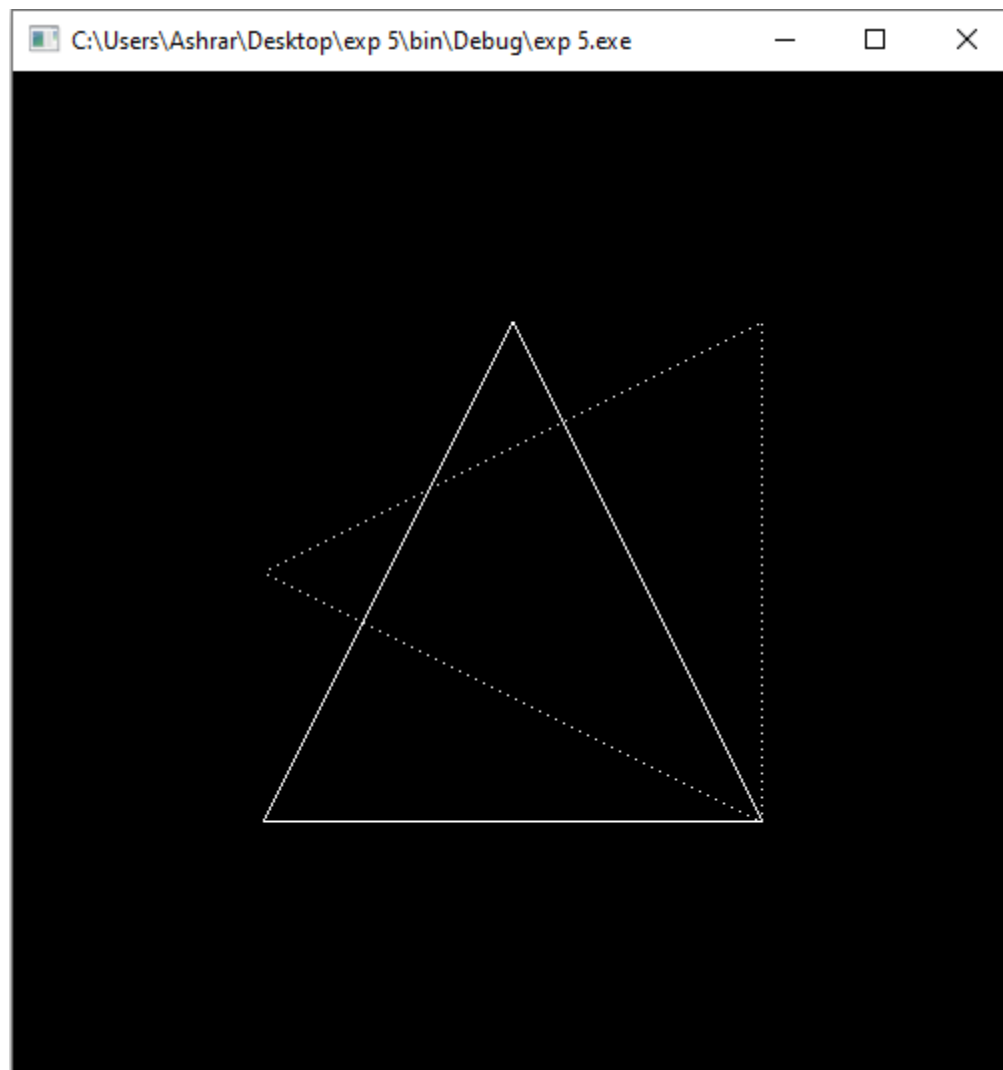
    glEnable (GL_LINE_STIPPLE);
    glLineStipple (1, 0x8888);
    glLoadIdentity ();
    glRotatef (90.0, 0.0, 0.0, 1.0);
    draw_triangle ();
    glDisable (GL_LINE_STIPPLE);

    glFlush ();
}

main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);
    glutInitWindowSize (500, 500);
    glutInitWindowPosition (100, 100);
    glutCreateWindow (argv[0]);
    init ();
    glutDisplayFunc(display);
    glutMainLoop();
}

```

```
return 0;  
}
```



6. Write a program to draw a circle using Bresenham algorithm.

```
#include<windows.h>

#include<stdio.h>

#include<stdlib.h>

#include<string.h>

#include<math.h>

#include<GL/gl.h>

#include<GL/glu.h>

#include<GL/glut.h>


//const int screenWidth = 640*2;

//const int screenHeight = 480*2;

//GLdouble A,B,C,D; // values for scaling and shifting


void myInit(void)
{
    glClearColor(1.0, 1.0, 1.0, 0.0); //background color is white
    glColor3f(0.0f, 0.0f, 255.0f); //drawing color is black
    glPointSize(4.0); //a dot is 2 by 2 pixels
    //glLineWidth(4.0); //a line is 4 times thicker


    //initialize view (simple orthographic projection)
    glMatrixMode(GL_PROJECTION);
```

```

glLoadIdentity();

//glOrtho(-1.0,1.0,-1.0, 1.0, -1.0, 1.0);

// A = screenWidth/4.0; //Set values for scaling and shifting
// B= 0.0;
// C=D=screenHeight/2.0;

//set the viewing coordinates
gluOrtho2D(0.0, 640.0, 0.0, 480.0);
}

void myDisplay(void)
{
    glClear(GL_COLOR_BUFFER_BIT); //clear the screen
    // glRecti(10,10,150,60); //draw a rectangle with background color

    glPointSize(2.0);

    GLint h = 200,k = 200, x, y, r, d;

    y = r = 100;
    d = 3 - 2*r;

    for(x=0; x<=y; x++)
    {

```

```
glBegin(GL_POINTS);  
{
```

```
glVertex2i(x+h,y+k);  
glVertex2i(x+h,k-y);  
glVertex2i(y+h,x+k);  
glVertex2i(y+h,k-x);  
glVertex2i(-y+h,x+k);  
glVertex2i(-y+h,k-x);  
glVertex2i(-x+h,y+k);  
glVertex2i(-x+h,k-y);
```

```
}  
glEnd();
```

```
if(d<0)
```

```
    d = d + 4*x + 6;
```

```
else{
```

```
    d = d + 4*(x-y) + 10;
```

```
    y--;
```

```
}
```

```
}
```

```
glFlush(); //send all output to display
```

```
}
```

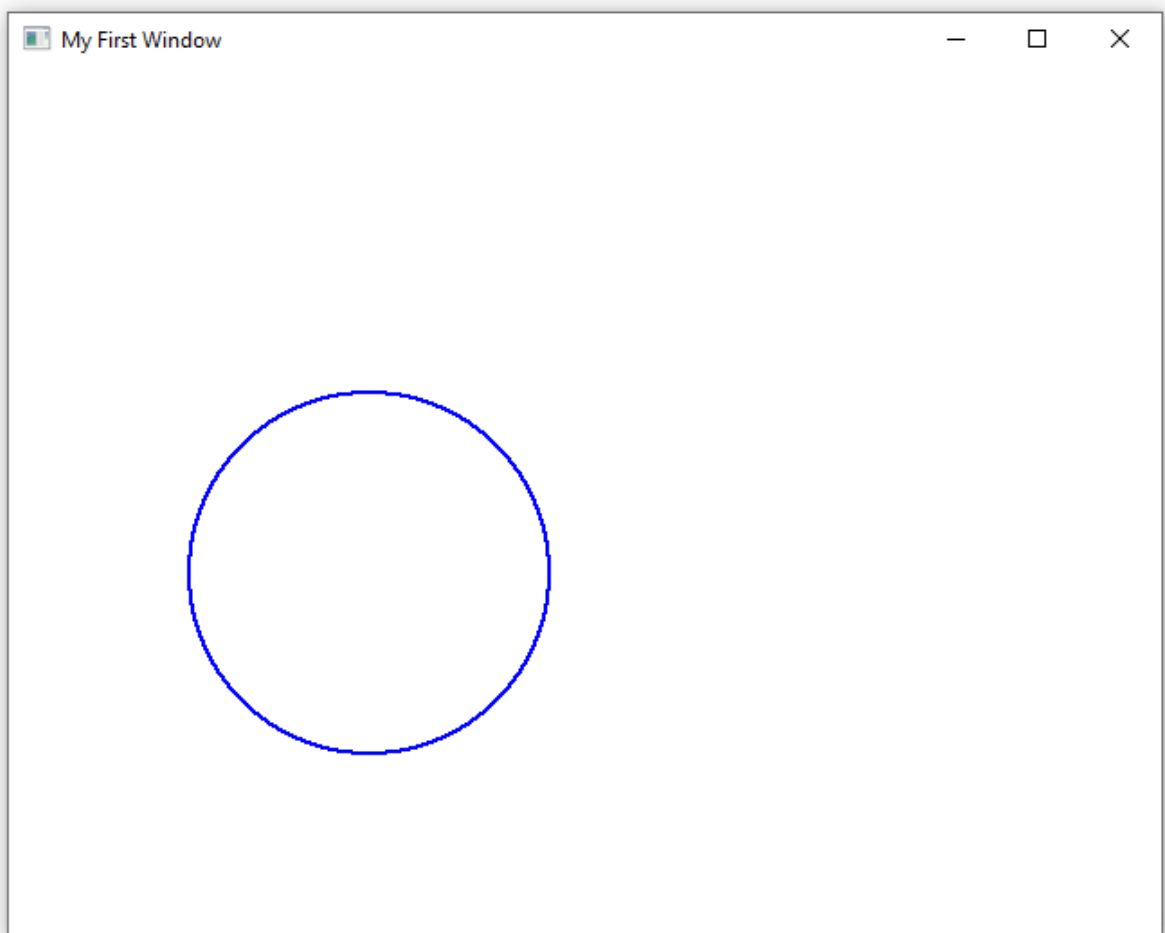
```
int main(int argc, char** argv)
{
    glutInit(&argc, argv); //initialize the toolkit
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB); //set the display mode
    glutInitWindowSize(640,480); //set window size
    //set window position on screen
    glutInitWindowPosition(100,150);
    //open the screen window and set the name
    glutCreateWindow("My First Window");

    //register the callback functions
    glutDisplayFunc(myDisplay);

    myInit();
    glutMainLoop(); //go into a perpetual loop

    return 1;

}
```



7. Write a program to show scaling transformation.

```
#include<windows.h>

#include <GL/glut.h>

#include <stdlib.h>

void init(void)

{

glClearColor (0.0, 0.0, 0.0, 0.0);

glMatrixMode(GL_PROJECTION);

glLoadIdentity();

glOrtho(-50.0, 50.0, -50.0, 50.0, -1.0, 1.0);

glMatrixMode(GL_MODELVIEW);

glLoadIdentity();

}

void draw_triangle(void)

{ glBegin (GL_LINE_LOOP);

    glVertex2f(0.0, 25.0);

    glVertex2f(25.0, -25.0);

    glVertex2f(-25.0, -25.0);

    glEnd();

}

void display(void)

{ glClear (GL_COLOR_BUFFER_BIT);

    glColor3f (1.0, 1.0, 1.0);

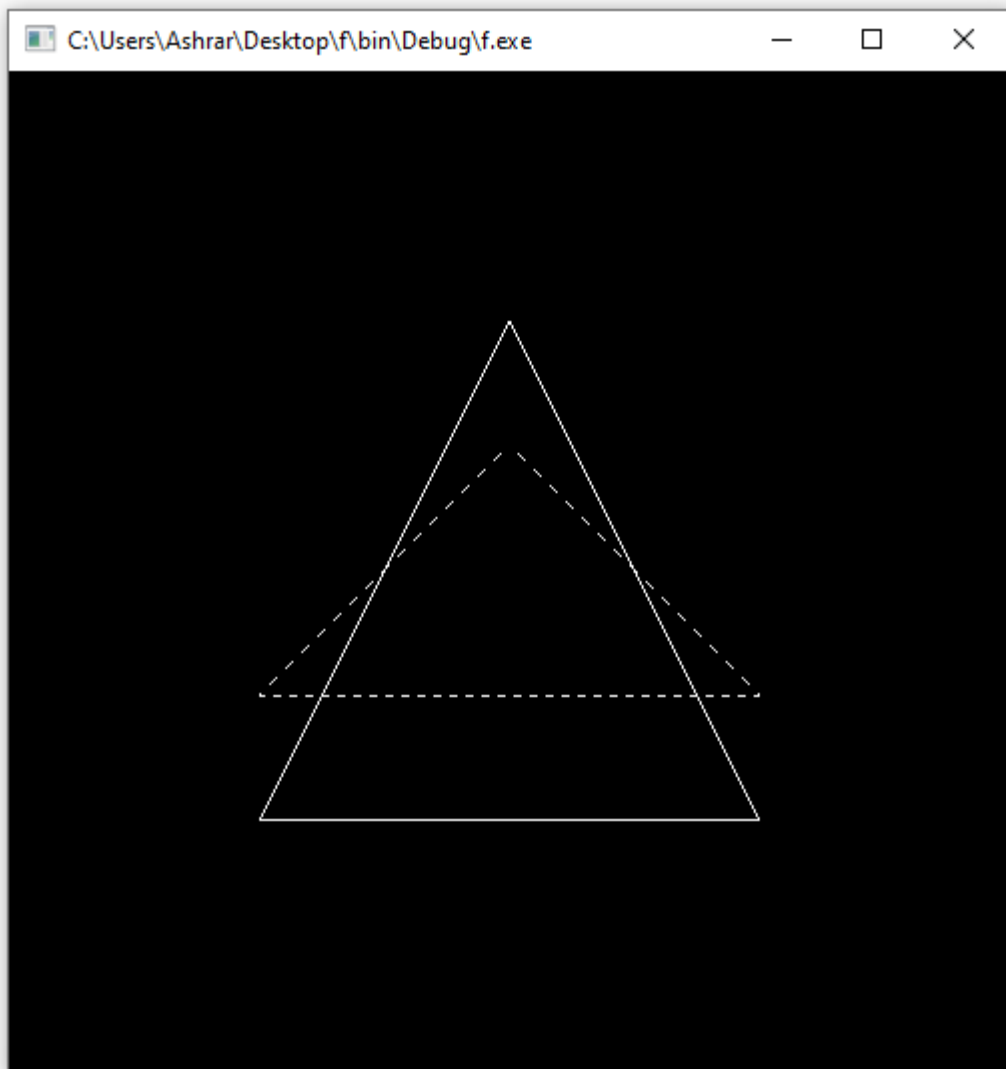
    glLoadIdentity ();

    glColor3f (1.0, 1.0, 1.0);
```



```
draw_triangle ();  
glEnable (GL_LINE_STIPPLE);  
  
glLineStipple (1, 0xF0F0);  
glLoadIdentity ();  
glScalef (1.0, 0.5, 1.0);  
draw_triangle ();  
glDisable (GL_LINE_STIPPLE);  
glFlush();  
  
}
```

```
int main(int argc, char** argv)  
{ glutInit(&argc, argv);  
  glutInitDisplayMode (GLUT_SINGLE | GLUT_RGB);  
  glutInitWindowSize (500, 500);  
  glutInitWindowPosition (100, 100);  
  glutCreateWindow (argv[0]);  
  init ();  
  glutDisplayFunc(display);  
  glutMainLoop();  
  return 0;  
}
```



8. Write a program to show mouse operation.

```
#include<windows.h>
```

```
#include<GL/glut.h>
```

```
void myInit(void)
```

```
{
```

```
    glClearColor(1.0, 1.0, 1.0, 1.0);
```

```
    glColor3f(0.0f, 0.0f, 0.0f);
```

```
    glPointSize(4.0);
```

```
    glMatrixMode(GL_PROJECTION);
```

```
    glLoadIdentity();
```

```
    glOrtho(-1.0, 1.0, -1.0, 1.0, -1.0, 1.0);
```

```
    gluOrtho2D(0.0, 640, 0.0, 480.0);
```

```
}
```

```
void myMouse(int button, int state, int x, int y)
```

```
{
```

```
    if(button == GLUT_LEFT_BUTTON && state == GLUT_DOWN)
```

```
    {
```

```
        glColor4f(0.0, 1.0, 0.0, 1.0);
```

```
        glRecti(200, 150, 500, 330);
```

```
        glFlush();
```

```
    }
```

```
    else if (button == GLUT_RIGHT_BUTTON && state == GLUT_DOWN)
```

```
    {
```

```
        glColor3f(1.0, 0.0, 0.0);
```

```

GLint h = 350, k = 240, x, y, r, d;

y = r = 60;

d = 3 - 2*r;

for(x=0; x<=y; x++)
{
    glBegin(GL_POINTS);
    {
        glVertex2i(x+h, y+k);
        glVertex2i(x+h, k-y);
        glVertex2i(y+h, x+k);
        glVertex2i(y+h, k-x);
        glVertex2i(-y+h, x+k);
        glVertex2i(-y+h, k-x);
        glVertex2i(-x+h, y+k);
        glVertex2i(-x+h, k-y);
    }
    glEnd();
    if(d<0)
        d = d + 4*x + 6;
    else
    {
        d = d + 4*(x-y) + 10;
        y--;
    }
}

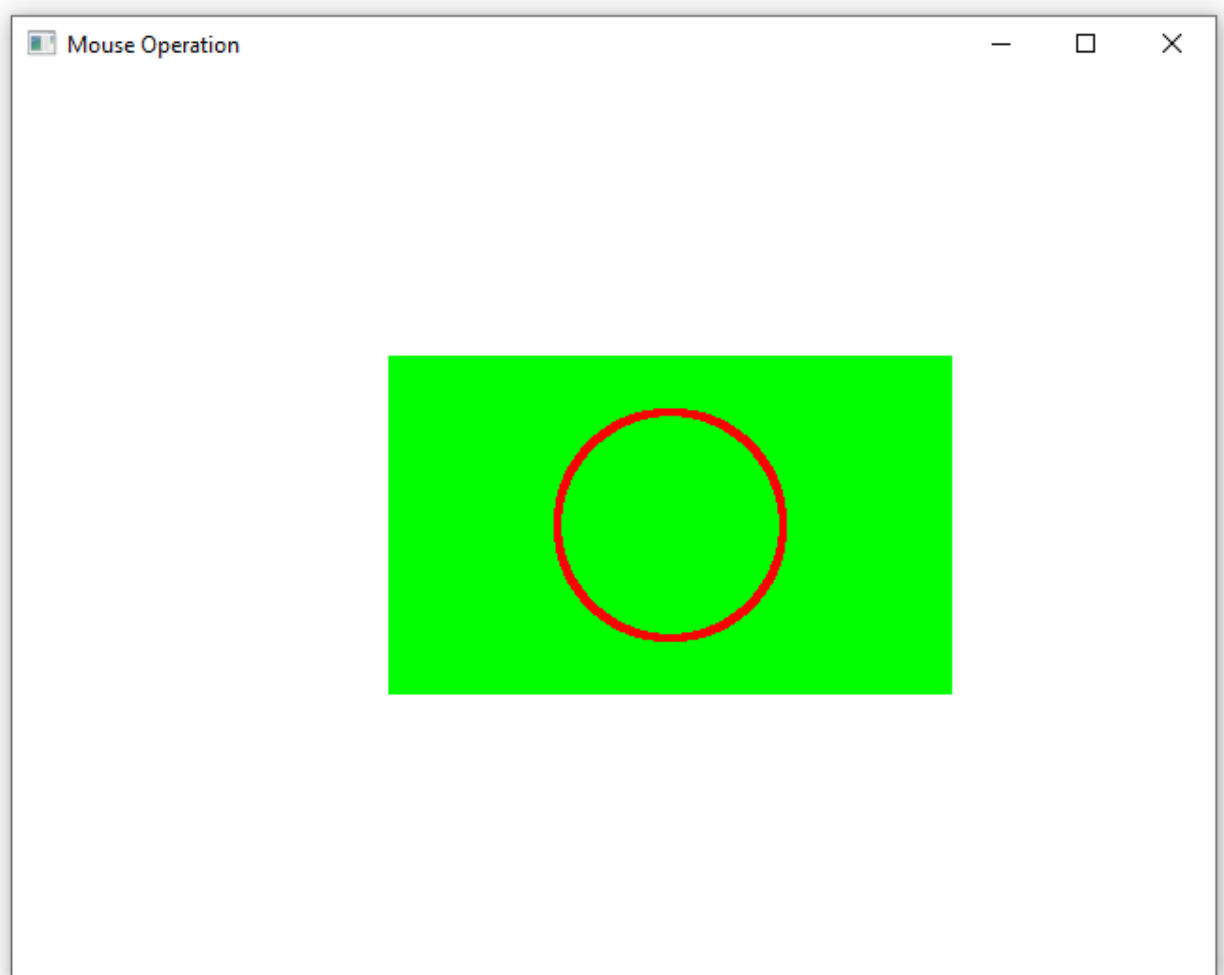
glFlush();

```

```
    }  
}
```

```
void myDisplay(void)  
{  
    glClear(GL_COLOR_BUFFER_BIT);  
    glFlush();  
}
```

```
int main(int argc, char** argv)  
{  
    glutInit(&argc, argv);  
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB);  
    glutInitWindowSize(640, 480);  
    glutInitWindowPosition(100, 150);  
    glutCreateWindow("Mouse Operation");  
    glutDisplayFunc(myDisplay);  
    glutMouseFunc(myMouse);  
  
    myInit();  
    glutMainLoop();  
  
    return 1;  
}
```



9. Write a program to show arm motion.

```
#include<windows.h>

# include <GL/gl.h>
# include <GL/glu.h>
# include <GL/glut.h>

static int shoulder = 0, elbow = 0;

void init()
{
    glClearColor(0.0,0.0,0.0,0.0);
}

void display(void)
{
    glClear(GL_COLOR_BUFFER_BIT);
    glPushMatrix();
    glTranslatef(-1.0,0.0,0.0);
    glRotatef((GLfloat) shoulder,0.0,0.0,1.0);
    glTranslatef(1.0,0.0,0.0);
    glPushMatrix();
    glScalef(2.0, 0.4, 1.0);
    glutWireCube(1.0);
    glPopMatrix();

    glTranslatef(1.0, 0.0, 0.0);
    glRotatef((GLfloat) elbow, 0.0, 0.0, 1.0);
    glTranslatef(1.0, 0.0, 0.0);
```

```
glPushMatrix();  
glScalef(2.0, 0.4, 1.0);  
glutWireCube(1.0);  
glPopMatrix();
```

```
glPopMatrix();  
glutSwapBuffers();  
}
```

```
void reshape (int w, int h)  
{  
    glViewport(0, 0, (GLsizei) w, (GLsizei) h);  
    glMatrixMode(GL_PROJECTION);  
    glLoadIdentity();  
    gluPerspective(65.0, (GLfloat) w / (GLfloat) h, 1.0, 20.0);  
    glMatrixMode(GL_MODELVIEW);  
    glLoadIdentity();  
    glTranslatef(1.0, 0.0, -5.0);  
}
```

```
void keyboard(unsigned char key, int x, int y)  
{  
    switch (key)  
    {  
        case 's':
```



```
    shoulder = (shoulder + 5) % 360;  
    glutPostRedisplay();  
    break;
```

```
case 'S':
```

```
    shoulder = (shoulder - 5) % 360;  
    glutPostRedisplay();  
    break;
```

```
case 'e':
```

```
    elbow = (elbow + 5) % 360;  
    glutPostRedisplay();  
    break;
```

```
case 'E':
```

```
    elbow = (elbow - 5) % 360;  
    glutPostRedisplay();  
    break;
```

```
case 27:
```

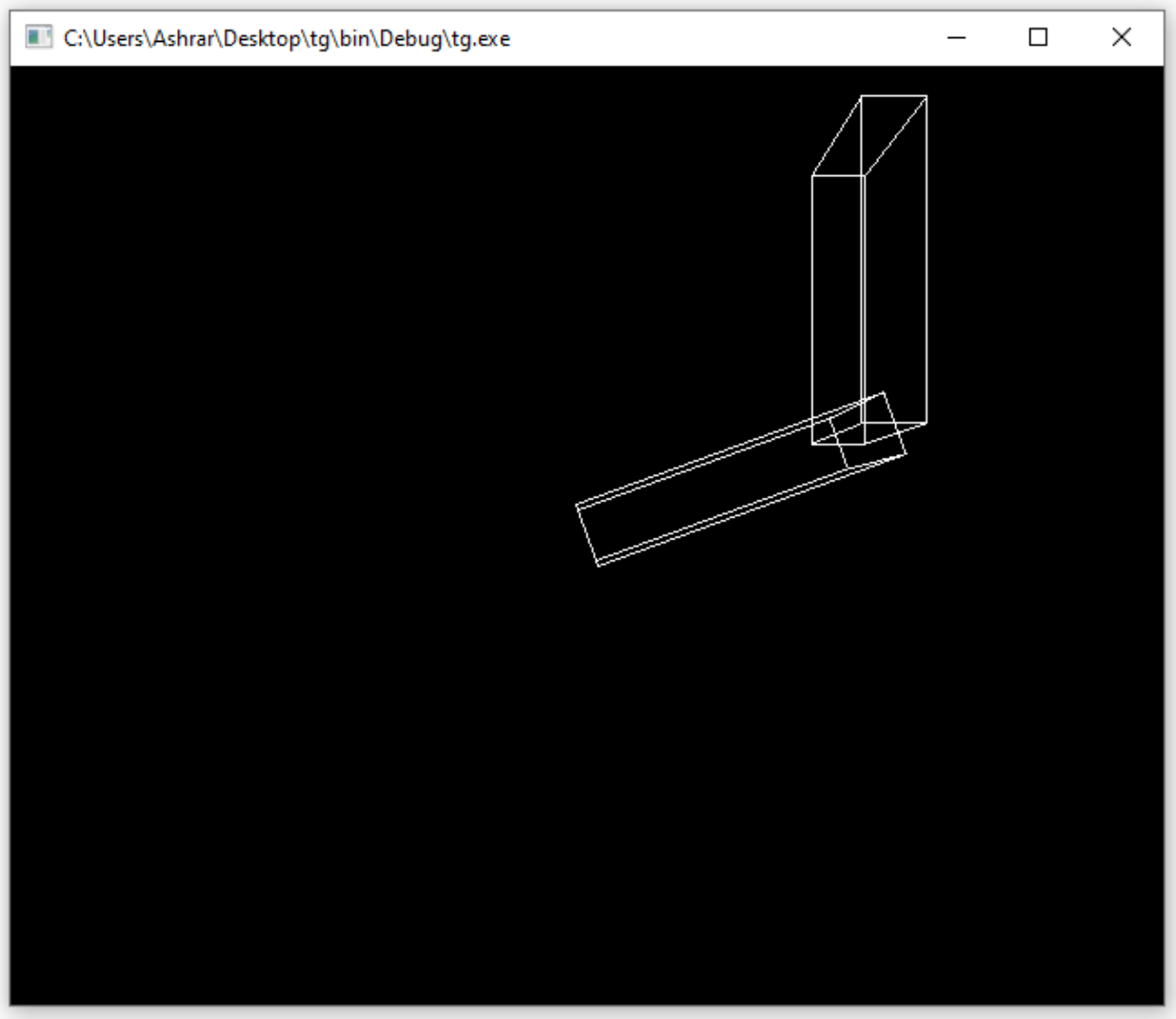
```
    exit(0);  
    break;
```

```
default:
```

```
    break;
```

```
}  
}
```

```
int main(int argc, char** argv)  
{  
    glutInit(&argc, argv);  
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB);  
    glutInitWindowSize(500,500);  
    glutInitWindowPosition(100, 100);  
    glutCreateWindow(argv[0]);  
    init();  
    glutDisplayFunc(display);  
    glutReshapeFunc(reshape);  
    glutKeyboardFunc(keyboard);  
    glutMainLoop();  
    return 0;  
}
```



10. Write a program to draw ellipse using Bresenham algorithm.

```
#include<windows.h>
```

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<string.h>
```

```
#include<math.h>
```

```
#include<GL/gl.h>
```

```
#include<GL/glu.h>
```

```
#include<GL/glut.h>
```

```
const int screenWidth = 640*2;
```

```
const int screenHeight = 480*2;
```

```
GLdouble A,B,C,D; // values for scaling and shifting
```

```
void myInit(void)
```

```
{
```

```
    glClearColor(1.0, 1.0, 1.0, 0.0); //background color is white
```

```
    glColor3f(0.0f, 255.0f, 0.0f); //drawing color is black
```

```
    glPointSize(4.0); //a dot is 2 by 2 pixels
```

```
    //glLineWidth(4.0); //a line is 4 times thicker
```

```
//initialize view (simple orthographic projection)
```

```
    glMatrixMode(GL_PROJECTION);
```

```
glLoadIdentity();
```

```
glOrtho(-1.0,1.0,-1.0, 1.0, -1.0, 1.0);
```

```
A = screenWidth/4.0; //Set values for scaling and shifting
```

```
B= 0.0;
```

```
C=D=screenHeight/2.0;
```

```
//set the viewing coordinates
```

```
gluOrtho2D(0.0, 640.0, 0.0, 480.0);
```

```
}
```

```
void myDisplay(void)
```

```
{
```

```
glClear(GL_COLOR_BUFFER_BIT); //clear the screen
```

```
// glColor(10,10,150,60); //draw a rectangle with background color
```

```
glPointSize(2.0);
```

```
GLint h = 300, k = 300, x, x2, y, a = 150, b = 25;
```

```
x2 = a;
```

```
for(x=1;x<=x2;x++)
```

```
{
```

```
y = b * sqrt(1 - (pow(x,2)/pow(a,2)));
```

```
glBegin(GL_POINTS);
```

```
{
```

```
glVertex2i(x+h,y+k);
```

```
glVertex2i(-x+h,-y+k);
```

```
glVertex2i(-x+h,y+k);
```

```
glVertex2i(x+h,-y+k);
```

```
}
```

```
glEnd();
```

```
}
```

```
glFlush(); //send all output to display
```

```
}
```

```
int main(int argc, char** argv)
```

```
{
```

```
glutInit(&argc, argv); //initialize the toolkit
```

```
glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB); //set the display mode
```

```
glutInitWindowSize(640,480); //set window size
```

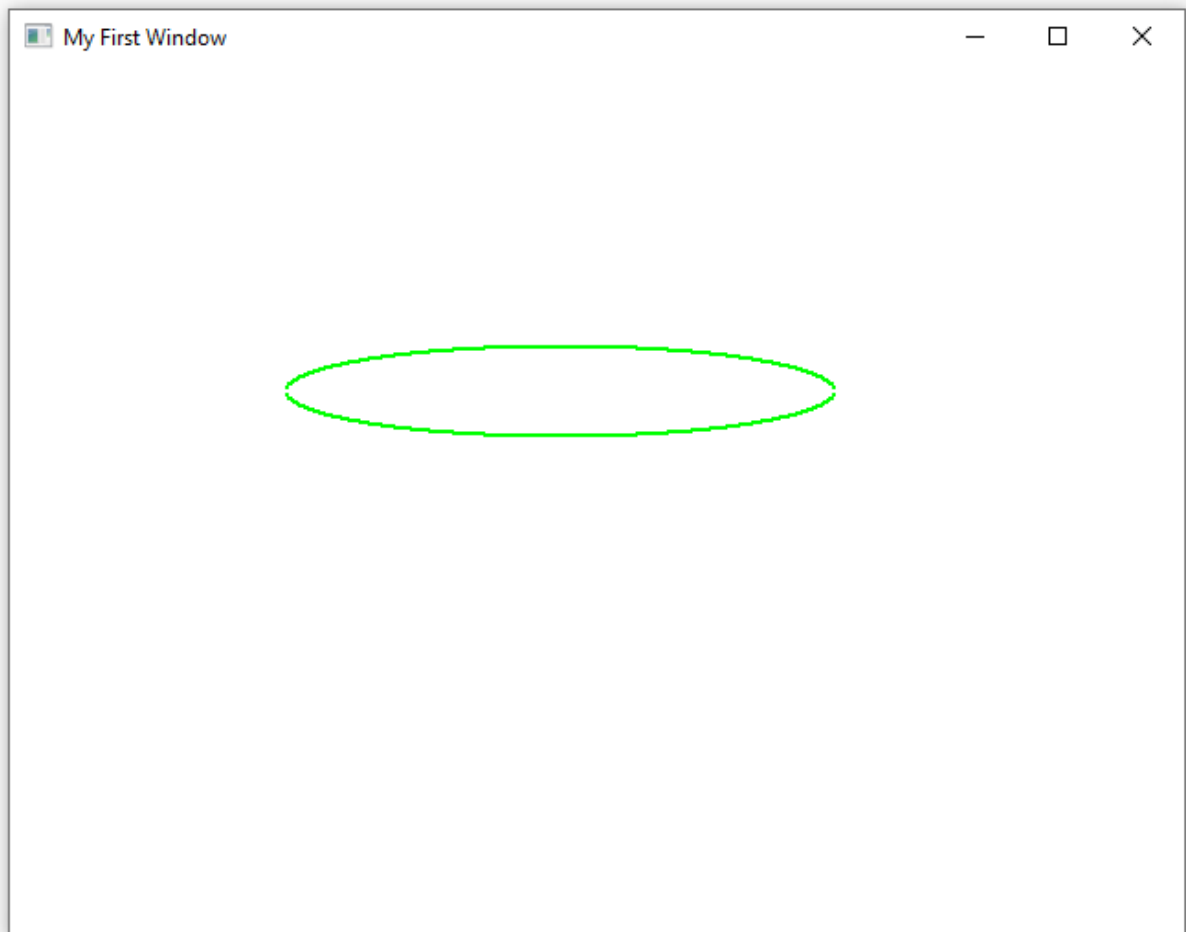
```
//set window position on screen
```

```
glutInitWindowPosition(100,150);
```

```
//open the screen window and set the name
```

```
glutCreateWindow("My First Window");
```

```
//register the callback functions  
glutDisplayFunc(myDisplay);  
  
myInit();  
glutMainLoop(); //go into a perpetual loop  
  
return 1;  
  
}
```



11. Write a program to draw a polygon.

```
#include<windows.h>

#include<stdio.h>

#include<stdlib.h>

#include<string.h>

#include<math.h>

#include<GL/gl.h>

#include<GL/glu.h>

#include<GL/glut.h>


const int screenWidth = 640;

const int screenHeight = 480;

GLdouble A,B,C,D; // values for scaling and shifting


void myInit(void)
{
    glClearColor(1.0, 1.0, 1.0, 0.0); //background color is white
    glColor3f(0.0f, 0.0f, 0.0f); //drawing color is black
    glPointSize(4.0); //a dot is 2 by 2 pixels
    //glLineWidth(4.0); //a line is 4 times thicker


    //initialize view (simple orthographic projection)
    glMatrixMode(GL_PROJECTION);
```



```
glLoadIdentity();
```

```
glOrtho(-1.0,1.0,-1.0, 1.0, -1.0, 1.0);
```

```
A = screenWidth/4.0; //Set values for scaling and shifting
```

```
B= 0.0;
```

```
C=D=screenHeight/2.0;
```

```
//set the viewing coordinates
```

```
gluOrtho2D(0.0, 640.0, 0.0, 480.0);
```

```
}
```

```
void myDisplay(void)
```

```
{
```

```
glClear(GL_COLOR_BUFFER_BIT); //clear the screen
```

```
// glRecti(10,10,150,60); //draw a rectangle with background color
```

```
/*glBegin(GL_POINTS); //draw three points
```

```
{
```

```
for(GLint x=0; x<50; x+=2){
```

```
    if(x<25)
```

```
        glVertex2i(100+x,x);
```

```
    else
```

```
        glVertex2i(x,x+100);
```

```
    }
```

```
}
```

```
glEnd();*/
```

```
glBegin(GL_POLYGON); //draw lines
```

```
{
```

```
    glColor3f(1.0,0.0,0.0);
```

```
glVertex2i(100,50);
```

```
glVertex2i(200,130);
```

```
glVertex2i(200,70);
```

```
glVertex2i(200,130);
```

```
glVertex2i(500,170);
```

```
glVertex2i(300,130);
```

```
}
```

```
glEnd();
```

```
glFlush(); //send all output to display
```

```
}
```

```
int main(int argc, char** argv)
```

```
{
```

```
    glutInit(&argc, argv); //initialize the toolkit
```

```
    glutInitDisplayMode(GLUT_SINGLE | GLUT_RGB); //set the display mode
```

```
glutInitWindowSize(640,480); //set window size
//set window position on screen
glutInitWindowPosition(100,150);
//open the screen window and set the name
glutCreateWindow("My First Window");

//register the callback functions
glutDisplayFunc(myDisplay);
//glutMouseFunc(myMouse);

myInit();
glutMainLoop(); //go into a perpetual loop

//fprintf(stderr, "Hello!\n");
return 1;

}
```



12. Write a program to show the line triangle transformation.

```
#include<windows.h>
#include<GL/gl.h>
#include<GL/glut.h>
void triangle()
{
    glColor3f(1.0,0.0,1.0);
    glBegin(GL_TRIANGLES);
    glVertex2i(50,150);
    glVertex2i(50,250);
    glVertex2i(130,200);
    glEnd();
}
void line(int sx,int sy,int dx,int dy)
{
    glColor3f(1.0,1.0,0.0) ;
    glBegin(GL_LINES);
    glVertex2i(sx,sy);
    glVertex2i(dx,dy);
    glEnd();
}
void display(void)
{
    glClear(GL_COLOR_BUFFER_BIT) ;
    line(100,150,100,250);
```

```

    triangle();
    glTranslatef(100.0,50.0,0.0);
    line(100,150,100,250);
    triangle();
    glFlush() ;
}

void init (void)
{
    glClearColor(1.0,1.0,1.0,0.0);
    gluOrtho2D(0.0,350.0,0.0,350.0);
}

int main(int argc,char** argv)
{
    glutInit(&argc,argv);
    glutInitDisplayMode(GLUT_SINGLE|GLUT_RGB);
    glutInitWindowSize(350,350);
    glutInitWindowPosition(100,100) ;
    glutCreateWindow ("TRANSFORMATION");
    init();
    glutDisplayFunc(display);
    glutMainLoop() ;
    return 0;
}

```

